

Chapter 5
Unit Testing

Concepts and Principles: Annotations, JUnit, metaprogramming, test coverage, test suites, unit testing;

Unit testing is a practice wherein we automatically execute our code to check that it does what we think it should. With unit testing, we can build a collection of tests to check that our expectations about how the code works conform to reality. In addition, these tests can be quickly executed at any point in the development process to confirm that the behavior of the code still meets the expectations we captured in our tests. Writing unit tests also provides insights into the quality of a design, because classes with many dependencies or objects with a complex state space will be difficult to test effectively. This chapter introduces mechanisms that facilitate unit testing (metaprogramming and unit testing frameworks) and presents basic techniques for designing unit tests and evaluating their quality.

Design Context

The first examples focus on the library function to compute the absolute value of an integer (`Math.abs(int)`). The remainder of the chapter discusses the testing of code derived from design elements of the Solitaire sample application, starting with a method of the `Suit` enumerated type. For simplicity, the later examples are slightly adapted from the actual project code. They revolve around `FoundationPile`, a class that represents one of the four piles where finished suits are accumulated in a game of Solitaire. The final design context concerns the testing of class `GameModel`, which encapsulates the entire aggregated state of a game in progress.

© Springer Nature Switzerland AG 2026

M. P. Robillard, *Introduction to Software Design with Java*,
https://doi.org/10.1007/978-3-032-11821-9_5

5.1 Introduction to Unit Testing

Software quality problems are often caused by programmers writing code that does not always do what they expect, and the programmers remaining ignorant of this mismatch between expectations and reality. To illustrate this issue as concretely as possible, while reviewing some of the concepts we have seen previously, let us consider the following method to retrieve an instance of a class `Card` that represents the playing card specified by a textual description.

```
/**
 * Returns the card that corresponds to the input description.
 *
 * @param pCardAsString Describes a card in the form
     "RANK of SUIT", where RANK is a valid rank name
 *   and SUIT is a valid suit name (case insensitive).
 * @return The card described by pCardAsString.
 * @throws IllegalArgumentException if the input does not
     describe a valid card in the required form.
 * @pre pCardAsString != null
 */
public static Card toCard(String pCardAsString) {
  String[] parts = pCardAsString.toUpperCase().split(" ");
  return Card.get(Rank.valueOf(parts[0]),Suit.valueOf(parts[2]));
}
```

This implementation first splits the input string in different parts as separated by whitespace using the `split` library method. Then, it relies on the `valueOf` method created for each enumerated type to retrieve the corresponding instance of the type (e.g., `"King"` becomes `Rank.KING`). Conveniently, method `valueOf` throws an `IllegalArgumentException` if it cannot convert the string to the name of an enum constant, which will propagate out of method `toCard` to fulfill that part of the specification.

At a glance, and with insufficient caffeine, this code looks reasonable. If we input any valid string, such as “King of Hearts”, we receive the corresponding card. And if we input strings with an invalid rank or suit, such as “X of Clubs”, we get the expected exception. Unfortunately, this simple-looking code is full of bugs. Among others, if the separator “of” is missing or mistyped, the method does not behave as expected.

One way to detect bugs, and to gain confidence that a part of the code does what we expect, is to test it. Testing is a software quality assurance technique that can take many forms. In this book, I focus on one specific testing approach called *unit testing*. The goal of unit testing is to test small parts of the code separately, in isolation. This way, if a test fails, we know where to look for problems. Writing unit tests also helps us think more rigorously about the behavior our code should have. For example, the specification of the method above is ambiguous about whether the qualifier “case insensitive” applies to the separator “of” or not, but to test this behavior we need a precise specification.

A *unit test* consists of one or more executions of a *unit under test* (UUT) with some input data and the comparison of the result of the execution against some *oracle*. A UUT is whatever part of the code we wish to test in isolation. In practice, UUTs are often individual methods, but in some cases they can also be entire classes, initialization statements, or certain paths through the code. The term *oracle* designates the expected result of the execution of a UUT.

For example, the statement: `Math.abs(-5) == 5;` technically qualifies as a test. In this case, the UUT is the library method `Math.abs(int)`, which computes the absolute value of its input. The input data is the integer literal `-5`, and the oracle is the value 5. The comparison of the result of executing the UUT with the oracle is called an *assertion*. The name captures the idea that the role of the comparison is to *assert* that the result is what we expect.

When testing instance methods, it is important to remember that the input data includes the *implicit argument* (the instance that receives the method call). As a second example that involves an implicit argument, let us consider a version of the `Suit` enumerated type that includes an additional method `sameColorAs(Suit)`. In a standard deck of cards, the Clubs and Spades suits are printed in black and the Diamonds and Hearts are printed in red.

```
public enum Suit {
  CLUBS, DIAMONDS, SPADES, HEARTS;

  public boolean sameColorAs(Suit pSuit) {
    assert pSuit != null;
    return (ordinal() - pSuit.ordinal()) % 2 == 0;
  }
}
```

With this design, we are not returning the color of a `Suit` instance, but rather, whether the suit is of the same color as some other `Suit` instance. In a game where the color of a suit does not matter, only whether it is the same color as a given suit, this design decision follows the principle of information hiding. Returning to our example, something to note is that the method was written by a programmer who favors compact code over clarity of intent and robustness. To give ourselves confidence that this works as expected despite the hackery, we can test the method by calling it with a specific input and comparing the result with the expected value:

```
public static void main(String[] args) {
  boolean expected = false;
  System.out.println(expected == CLUBS.sameColorAs(HEARTS));
}
```

This example makes it clear that although method `sameColorAs` takes a single *explicit* argument, there are in fact two arguments to the UUT: the explicitly provided argument (`Suit.HEARTS`), and the implicit argument: `Suit.CLUBS`. According to the definition of a unit test provided above, this `main` method qualifies as a unit test: it includes a UUT (`Suit#sameColorAs`), some input data (`CLUBS` and `HEARTS`), an oracle (`false`), and an assertion that compares the result with the oracle. Executing the `main` method will tell us whether the test passes or not.

If we wanted to increase the number of inputs we test for `sameColorAs`, we could add additional pairs of suits. Because there are only four suits, we could test *all possible inputs* for `sameColorAs` with a mere 16 tests. This achievement, called *exhaustive testing*, is rarely possible (see Section 5.9). However, in the present case, writing an exhaustive test is trivial and executing it will show that method `sameColorAs` works correctly for all possible inputs. This is good news, but only ephemerally so, because source code is not set in stone. To continue with our scenario, let us say that the code is later changed to reorder the suits as follows without updating the `sameColorAs` method:

```
CLUBS, SPADES, DIAMONDS, HEARTS;
```

In this case, running the test again will immediately reveal a bug introduced by the fact that `sameColorAs` relies on an undocumented and unchecked assumption about the order of enumerated values. This example illustrates the second major benefit of unit tests: in addition to helping detect bugs in new code, they can also check that tested behavior that used to meet some specific expectation still does meet that expectation even after the code changes. Running tests to confirm that previous expectations about the behavior of the code are still respected, or to identify deviations from expected behavior caused by changes, is called *regression testing*.

Finally, an important observation about testing relates to what it *cannot* do. Testing cannot *verify* code to be correct. When a test passes, it only shows that the specific execution of the code that is being tested behaves as expected. There are software engineering techniques designed to provide certain guarantees about all possible code executions for a specific code element, but testing is not one of them. Section 5.9 provides further explanations of why testing is not a verification technique.

5.2 Unit Testing Framework Fundamentals with JUnit

Although it is possible to test a system manually, unit testing is normally done automatically. Because in software development the way to automate anything is to write code to do it, to automate software testing we also write code to test other code. This task is typically supported by a *unit testing framework*. The example with the `main` method in the previous section, although it qualifies as automation, was only for illustration purposes and is not a recommended way to test production code.

Unit testing frameworks automate a lot of the mundane aspects of unit testing, including collecting tests, running them, and reporting the results. In addition to tools to collect and run tests and display the results, frameworks also include a set of constructs to allow developers to write tests in a structured way and, if they so choose, efficiently. The major constructs supported by testing frameworks are *test cases*, *test suites*, *test fixtures* and *assertions*. The dominant unit testing framework for Java is called *JUnit*. I will introduce the basics of this framework sufficiently to

illustrate all of the testing techniques covered in this chapter. However, this chapter is not a tutorial on JUnit, and the Further Reading section provides pointers to additional information and coaching on how to use JUnit.

In JUnit, a unit test maps to a method. The code below illustrates a series of simple unit tests with JUnit.

```
public class AbsTest {
  @Test
  void testAbs_Positive() {
    assertEquals(5, Math.abs(5));
  }

  @Test
  void testAbs_Negative() {
    assertEquals(5, Math.abs(-5));
  }

  public void testAbs_Min() {
    assertEquals(Integer.MAX_VALUE,Math.abs(Integer.MIN_VALUE));
  }
}
```

The `@Test` *annotation instance* indicates that the annotated method should be run as a unit test. Section 5.4 explains annotations in more detail. For now, it suffices to say it is a marker we put in the code to indicate that the method is a unit test.[1] The code example above shows a *test class* that defines three tests, all intended to test the library method `Math.abs(int)`.

To constitute proper tests, test methods should contain at least one execution of a unit under test. The way to automatically verify that the execution of a unit under test has the expected effect is to call *assert methods*. Assert methods are different from the `assert` statement. They are declared as static methods of the JUnit `Assertions` class and all they do is test a predicate and, if the predicate is false, report a *test failure*. The JUnit framework includes a component called a *test runner*, which automatically scans some input code, detects all the tests in the input, executes them, and then reports whether the tests passed or failed. Figure 5.1 shows a screenshot of the result of executing all the tests in the test class `AbsTest` (above), using a version of the test runner available through the Eclipse integrated development environment. Two tests passed but one test, `testAbs_Min`, failed. Perhaps it seems surprising that the absolute value of `Integer.MIN_VALUE` is not, in fact, `Integer.MAX_VALUE`. This quirk is explained in the documentation for the method:

> Note that if the argument is equal to the value of Integer.MIN_VALUE, the most negative representable int value, the result is that same value, which is negative.

The reason for this design choice is imposed by the convention used to encode integers in Java, which allows for an extra negative number compared to the corresponding positive numbers. In addition to their bug detection potential, unit tests are a great way to surface corner cases.

[1] The definition of the `@Test` annotation is defined in the JUnit library, which must be added to a project's class path before it can be used. This chapter is based on JUnit version 5.

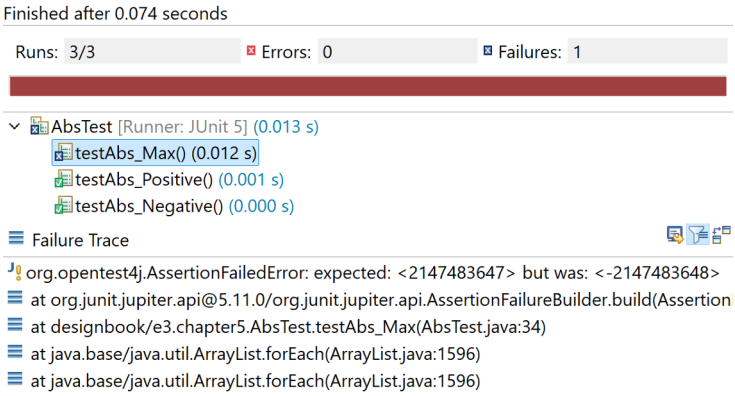


Fig. 5.1 Result of running JUnit using class `AbsTest` in the Eclipse IDE

5.3 Organizing Test Code

A collection of tests for a project is known as a *test suite*. By default, a project's test suite consists of all the unit tests for the production code in the project. However, it may sometimes be desirable to run only a certain subset of the tests (for example, to focus on a specific feature, or to save some time). Unit testing frameworks provide mechanisms to define arbitrary tests suites or, more generally, to run certain subsets of unit tests. As one example, JUnit provides a `@Suite` annotation that allows a developer to list a number of test classes to be executed together. As another example, the JUnit plug-in for the Eclipse IDE allows users to execute the tests for only one package, or even a single test class or method. Because executing tests is a concern somewhat independent from the issue of designing them, the rest of the chapter focuses on writing the tests themselves.

A common question when building a suite of unit tests is how to organize our tests in a sensible manner. There are different approaches, but in Java a common idiom is to have one test class per project class, where the test class collects all the tests that test methods or other usage scenarios that involve the class. Furthermore, it is common practice to locate all the testing code in a different source folder with a package structure that mirrors the package structure of the production code. The rationale for this organization is that, in Java, classes with the same package name are in the same *package scope* independently of their location in a file system. This means that classes and methods in the test package can refer to non-public (but non-private) members of classes in the production code, while still being separated from the production code. Figure 5.2 illustrates this practice. The figure shows a sample of tests in a `test` folder that match corresponding classes in the `src` folder. To emphasize the connection between the two, a common convention is to name the test class with the name of the production class it tests, followed by the suffix `Test`.

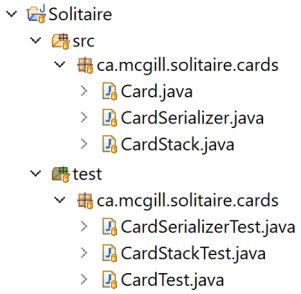


Fig. 5.2 Test suite organization for the Solitaire sample application

5.4 Metaprogramming

In the previous section, we saw that to indicate that a method is a test, we annotate it with the string `@Test`. The unit testing framework can then rely on this annotation to detect which methods are tests, and proceed to execute these methods as part of the execution of the test runner. This approach is special in that it requires the code to manipulate other code. Specifically, the testing framework first scans the code to detect tests, and then executes the code, without having any code that calls the test methods explicitly by name. This strategy is an illustration of a general programming feature called *metaprogramming*. Metaprogramming is the production of code that operates on a representation of a program's code. Although it may seem confusing at first, metaprogramming is just a special case of general-purpose programming. When we write code, this code typically operates on data that represents various things in the world (playing cards, geometric shapes, bank records, etc.). With metaprogramming, this data happens to be pieces of software code (classes, methods, fields, etc.). Although metaprogramming is a programming feature, it is instrumental for testing, and can be used to implement many design ideas. In Java, metaprogramming is called *reflection*, and library support for metaprogramming features is available through the class `java.lang.Class` and the package `java.lang.reflect`.

Introspection

The most basic metaprogramming task is to obtain a reference to an object that represents a piece of code to learn about it, a procedure called *introspection*. In Java, the class `Class<T>` is the main access point for metaprogramming. For example, to obtain a reference to an object that represents the `String` class, we can do:

```
try {
  String fullyQualifiedName = "java.lang.String";
  Class<String> stringClass =
    (Class<String>) Class.forName(fullyQualifiedName);
} catch (ClassNotFoundException e) {
  e.printStackTrace();
}
```

The call to `Class.forName` takes as input the fully-qualified name of the `String` class and returns a reference to an instance of class `Class` that represents class `String`, as illustrated in the object diagram of Figure 5.3.

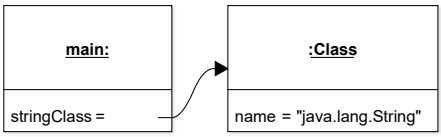


Fig. 5.3 Instance of class `Class` that represents class `String`

Calls to method `forName` are brittle because the requested class may not exist. In the example, the call is enclosed in a `try-catch` block. Method `forName` declares the checked exception `ClassNotFoundException` and throws it whenever the argument does not correspond to the fully-qualified name of a class on the class path. This may seem easy to prevent in the example above, given the use of a string literal to specify the name of the argument to `forName`. However, any string can be supplied as argument to `Class.forName`, so the requested class name may not be known at compile time, as in the example below:

```
public static void main(String[] args) {
  try {
    Class<?> theClass = Class.forName(args[0]);
  }
  catch (ClassNotFoundException e) {
      /* ... */
  }
}
```

For the same reason, we have to use a *type wildcard* as the instance of the type parameter in the type declaration of the variable that receives the reference supplied by `forName`. In the last example, the variable is declared as `Class<?>`. The exact functioning of the type wildcard is outside the scope of this book, but for now it suffices to say that it acts as a placeholder for any type. In the previous example, because we know exactly which type parameter is appropriate for class `Card`, we can use a downcast instead.

Besides the `forName` library method, Java offers two other ways to obtain a reference to an instance of class `Class` that are less brittle: *class literals*, and through an instance of the class of interest. Both strategies are illustrated in the code below:

```
Class<String> stringClass1 = String.class;
String someString = "Hello, World!";
Class<?> stringClass2 = someString.getClass();
System.out.println(stringClass1 == stringClass2);
```

The first line shows the use of class literals. In Java, a *class literal* is a literal expression that consists of the name of a class followed by the suffix `.class`, and that refers to a reference to an instance of class `Class` that represents the class named before the suffix. So, `String.class` refers to the instance of class `Class` that represents class `String`. Because, in the case of class literals, the argument `T` to the

type parameter of `Class<T>` is guaranteed to be known at compile time, we can include it in the variable declaration. Class literals are the least brittle way to obtain a reference to an instance of class `Class`, but they require that we know the exact class to introspect at compile time.

The final way to obtain an instance of class `Class` is through an instance of the class, as illustrated in the second and third lines of the code fragment. As will be explained in detail in Chapter 7, it is possible to call method `getClass()` on any object in a Java program, and the method will return a reference to an instance of class `Class` that represents the run-time type of the object. Because of polymorphism, this type may not be known at compile time, so in this case also we have to use the type wildcard in the declaration of the variable.[2] However, because any call to `getClass()` is guaranteed to return a valid reference to an instance of `Class`, the method does not declare to throw an exception.

The last line in the code fragment illustrates a very important property of class `Class`: its instances are unique (see Section 4.6). If executed, the code should always print `true` on the console. Indeed, class `Class` has no accessible constructor, and its instances can be considered to be unique flyweight objects (see Section 4.7).

With metaprogramming, we can introspect any class, including class `Class`. This may at first seem contrived, but it is actually not a special case: class `Class` is just another class. The following code:

```
Class<Class> classClass = Class.class;
```

will produce the object graph illustrated in the diagram in Figure 5.4.

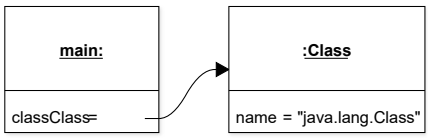


Fig. 5.4 Object graph for an instance of `Class.class`

Obtaining an instance of class `Class` is only the first step for introspection. The interface to class `Class` provides numerous methods that can be called to obtain objects that represent the members of the class, its superclass, etc. As one example of endless possibilities, the following code fragment prints the name of all the methods declared in class `String`. This example makes use of class `Method`, a library class intended to represent methods in Java code. Similar classes exist to represent constructors (`Constructor`) and fields (`Field`).

```
for (Method method : String.class.getDeclaredMethods()) {
  System.out.println(method.getName());
}
```

[2] In this case it is possible to use a type bound, e.g., `Class<? extends String>`.

Program Manipulation

Obtaining information about a code base, or code introspection, only constitutes the most basic form of metaprogramming. We may also want to manipulate the code of a program. Unlike more dynamic languages, Java does not allow adding or removing members from classes (and objects, by extension). However, it is possible to use metaprogramming features to change the accessibility of class members, set field values, instantiate objects, and invoke methods. I only provide a small overview of the features most relevant to software testing and design in general. The API documentation of the relevant library classes will provide the catalog of possibilities.

For this example, we will return to the version of class `Card` seen in Section 4.7, namely a class that realizes the FLYWEIGHT pattern and has a private constructor to prevent the creation of duplicate playing cards. Now we will use metaprogramming to get around the pattern and create a duplicate Ace of Clubs.

```
try {
  Card card1 = Card.get(Rank.ACE, Suit.CLUBS);
  Constructor<Card> cardConstructor =
    Card.class.getDeclaredConstructor(Rank.class, Suit.class);
  cardConstructor.setAccessible(true);
  Card card2 = cardConstructor.newInstance(Rank.ACE, Suit.CLUBS);
  System.out.println(card1 == card2);
} catch (ReflectiveOperationException e) {
  e.printStackTrace();
}
```

In this example, the second statement obtains a reference to an instance of class `Constructor` that represents the (private) constructor of class `Card`. To make calls to this constructor *accessible* in a scope outside its class, the third statement changes the accessibility of the constructor, effectively bypassing the `private` keyword in the code. The fourth statement calls `newInstance` on the `Constructor` object. This call makes a new instance of the declaring class of the constructor *represented* by the `Constructor` instance, as opposed to a new instance of class `Constructor`. As this is class `Card`, and the constructor of this class requires two arguments of type `Rank` and `Suit`, we pass values of these types to the `newInstance` call. Because a lot of things can go wrong with a reflective constructor invocation, most of the methods in this example declare to throw checked exceptions of various types, the supertype of which is `ReflectiveOperationException`.

Annotations as Program Metadata

With metaprogramming, it is possible for code to operate not only on data that consists of code elements (e.g., classes, methods, fields), but also on metadata *about* these code elements. In Java, it is possible to attach additional information (i.e., meta-information) to code elements in the form of *annotations*. We have already seen one type of annotation: the use of `@Test` to indicate that a method is a unit test in JUnit. An *annotation type* is declared similarly to an interface, for example:

```
public @interface Test {}
```

Then, *annotation instances* can be added to the code, in the form `@Test`. The main advantage of annotations is that they are typed and checked by the compiler. The `@Test` annotation used to flag unit tests in JUnit is thus a type annotation provided by the JUnit library, and its use is checked by the compiler. Annotations support many other features (see Further Reading), but their main usage scenario is to provide a way to add structured, type-checked metadata to some code elements, that can then be read by the compiler, development environments, unit testing frameworks, and similar tools. We will see other example applications of annotations later in the book. Because they are officially part of the code, information about annotations can also be accessed through metaprogramming.[3]

5.5 Structuring Tests

Writing unit tests for non-trivial classes is often a challenging creative process, not unlike writing production code. For this reason, there is no standard formula or template for writing the code of a unit test. In fact, browsing the test suites of different open-source projects will show that different communities follow different styles and use different testing techniques. This being said, certain basic principles are generally agreed upon, including that unit tests should be *fast, independent, repeatable, focused, and readable* [8].

- **Fast.** Unit tests are intended to be run often, and in many cases within a programming-compilation-execution cycle. For this reason, whatever test suite is executed should be able to complete in the order of a few seconds. Otherwise, developers will be tempted to omit running them, and the tests will stop being useful. This means that unit tests should avoid long-running operations such as intensive device I/O and network access, and leave the testing of such functionality to tests other than unit tests. These could include, for example, *acceptance tests* or *integration tests*.
- **Independent.** Each unit test should be able to execute in isolation. This means that, for example, one test should not depend on the fact that another test executes before to leave an input object in a certain state. First, it is often desirable to execute only a single test. Second, just like code, test suites evolve, with new tests being added and (to a minimum extent) some tests being removed. Test independence facilitates test suite evolution. Finally, JUnit and similarly-designed testing frameworks do not guarantee that tests will be executed in a predictable order. In practice, this means that each test should start with a fresh initialization of the state used as part of the test.

[3] However, only annotation instances of annotation types marked with the `@Retention(value=RUNTIME)` meta-annotation can be accessed in this way. See Further Reading for a reference to complementary information on annotations that covers meta-annotations.

- **Repeatable.** The execution of unit tests should produce the same result in different environments (for example, when executed on different operating systems). This means that test oracles should not depend on environment-specific properties, such as display size, CPU speed, or system fonts.
- **Focused.** Unit tests should exercise a slice of code execution behavior that is as narrow as possible. The rationale for this principle is that the purpose of unit tests is to help identify faults. If a unit test comprises 500 lines of code and tests a whole series of complex interactions between objects, it will not be easy to determine what went wrong if it fails. In contrast, a test that checks a single input on a single method call will make it easy to home in on a problem. Some have even argued that unit tests should comprise a single assertion [8]. My opinion is that in many cases this is too strict and can lead to inefficiencies. However, tests should ideally focus on testing *only one aspect* of *one unit under test*. If that unit under test is a method, we can refer to it as the *focal method* for the test.
- **Readable.** The structure and coding style of the test should make it easy to identify all the components of the test (unit under test, input data, oracle), as well as the rationale for the test. Are we testing the initialization of an object? A special case? A particular combination of values? Choosing an appropriate name for the test can often help in clarifying its rationale.

For example, let us write some unit tests for a method `canMoveTo` of a hypothetical class `FoundationPile` that could be part of the design of the Solitaire example application. The method should return `true` only if it is possible to move the input `pCard` to the top of the pile that an instance of the class represents. According to the rules of the game, this is only allowed if the pile is empty and the input card is an ace, or if the input card is of the same suit as the top of the pile, and of a rank immediately above the rank of the card at the top of the pile (e.g., you can only place a Three of Clubs on top of a Two of Clubs).

```
public class FoundationPile {
  public boolean isEmpty() { ... }
  public Card peek() { ... }
  public Card pop() { ... }
  public void push(Card pCard) { ... }

  public boolean canMoveTo(Card pCard) {
    assert pCard != null;
    if (isEmpty()) {
      return pCard.rank() == Rank.ACE;
    }
    else {
      return pCard.suit() == peek().suit() &&
        pCard.rank().ordinal() ==
          peek().rank().ordinal() + 1;
    }
  }
}
```

As our first test, we will keep things small and only test for the case where the pile is empty and the result should be false:

```
public class FoundationPileTest {
  @Test
  void testCanMoveTo_Empty_ReturnsFalse() {
    FoundationPile emptyPile = new FoundationPile();
    Card threeOfClubs = Card.get(Rank.THREE, Suit.CLUBS);
    assertFalse(emptyPile.canMoveTo(threeOfClubs));
  }
}
```

This test respects our five desired properties. It will execute with lightning speed, be independent from any other test that could exist, and is not affected by any environment properties. It is also focused, not only on a single method, but also on a specific input and return value combination for the method. Finally, many properties of this test add to its readability. First, the *name of the test* encodes the name of the focal method, the input of interest (an empty pile) and the expected value (false). Second, the *names of the variables* describe their content. Finally, the assertion statement is self evident. Reading the last line of the test, we see that calling `canMoveTo` with a Three of Clubs on an empty pile is expected to return false. We can then use the same clean structure to create a test for the case where `canMoveTo` should return `true` on an empty pile simply by replacing the input card with an ace, changing the assertion method to `assertTrue`, and updating the name of the test in consequence.

Test Fixtures

So far, we are only testing cases that involve an empty pile, which has limited *coverage*. In other words, our tests only exercise the true branch of the *if* statement, so part of the method's logic remains untested. This issue will be further discussed in Section 5.9. For now, let us mitigate this limitation by writing an additional test in the same class

```
@Test
void testCanMoveTo_NotEmptyAndSameSuit_ReturnsFalse() {
  FoundationPile pile = new FoundationPile();
  Card aceOfClubs = Card.get(Rank.ACE, Suit.CLUBS);
  pile.push(aceOfClubs);
  Card threeOfClubs = Card.get(Rank.THREE, Suit.CLUBS);
  assertFalse(pile.canMoveTo(threeOfClubs));
}
```

This test improves the test suite by adding to the coverage. However, we already note a lot of redundant code between this test and the previous ones we have written, including the code to instantiate `FoundationPile` and the code to create cards. If we had, say, 20 tests, this would add up to a lot of clutter. In test classes that group multiple test methods, it will often be convenient to define a number of default objects or values to be used as receiver objects, explicit parameters, and/or oracles. This practice helps limit the problem of DUPLICATED CODE† in test classes.

Baseline objects used for testing are often referred to as a *test fixture*, and declared as fields of a test class. However, for the reasons discussed above, and in particular because JUnit provides no ordering guarantee for test execution, it is crucial to preserve test independence. This implies that no test method should rely on the fixture being left in a given state by another test. Conveniently, JUnit 5 will, by default, instantiate a fresh version of the test class before running any test method. For this reason, the values of the fields of the test class will contain their initial value when any test executes.[4] Of course, immutable objects do not need to be reinitialized, so they can be stored as static fields of the class. The code below shows an improved version of our test class, which now uses a test fixture.

```
public class FoundationPile Test {
  private static final Card ACE_CLUBS =
    Card.get(Rank.ACE, Suit.CLUBS);
  private static final Card TWO_CLUBS =
    Card.get(Rank.TWO, Suit.CLUBS);
  private static final Card THREE_CLUBS =
    Card.get(Rank.THREE, Suit.CLUBS);

  private final FoundationPile aPile = new FoundationPile();

  @Test
  void testCanMoveTo_Empty_ReturnsFalse() {
    assertFalse(aPile.canMoveTo(THREE_CLUBS));
  }

  @Test
  void testCanMoveTo_Empty_ReturnsTrue() {
    assertTrue(aPile.canMoveTo(ACE_CLUBS));
  }

  @Test
  void testCanMoveTo_NotEmptyAndSameSuit_ReturnsFalse() {
    aPile.push(ACE_CLUBS);
    assertFalse(aPile.canMoveTo(THREE_CLUBS));
  }
}
```

This code not only avoids duplication, but also increases the readability of the tests by decluttering them. The only regression in test readability is due to the fact that, by using a field to refer to the pile, we lose our flexibility to name the pile with a variable name that describes its state. In this context, this is a small price to pay for the benefit of using the fixture. Although it would always be possible to alias the `aPile` field into an appropriately named variable (e.g., `emptyPile`), it is not clear that this would necessarily improve readability, because aliasing can also hinder code comprehension.

[4] As an alternative, it is possible to reuse an instance of the test class for multiple tests, but to nominate a method of the test class to execute before any test method, and initialize all the required structures afresh. These features are available in JUnit via different annotations.

Code Exploration: Solitaire · FoundationsTest

Testing an aggregate class in practice.

Most of the examples in this chapter are simplified versions adapted from the test suite of the Solitaire sample application. In Solitaire, class `FoundationsTest` provides a complete example of test code for a collection of stack-like structures. Instead of a `FoundationPile` class with four instances, the design involves a single `Foundations` class that stores the four piles in one object and indexes each pile using the enumerated type `FoundationPile`. The rest is very similar to the example in the chapter. Test class `FoundationsTest` includes tests for `canMoveTo` that achieve complete branch coverage. Because the class under test, `Foundations`, makes heavy use of design by contract, there is no need for testing exception handling behavior.

5.6 Testing Exceptional Conditions

An important point when writing unit tests is that what we are testing is that the unit under test does what it is expected to. This means that when using design by contract, it does not make sense to test code with input that does not respect the method's preconditions, because the resulting behavior is unspecified. For example, let us consider a version of method `peek` of class `FoundationPile` (introduced in the previous section) which returns the top of the pile.

```
class FoundationPile {
  boolean isEmpty() { ... }
  /*
   * @return The card on top of the pile.
   * @pre !isEmpty()
   */
  Card peek() { ... }
}
```

The documented precondition implies that the method cannot be expected to fulfill its contract (to return the top card) if the precondition is not met. Thus, if we call the method on an empty pile, there is no expectation to test. The situation is different, however, when raising exceptions is explicitly part of the interface. Let us consider the following slight variant of method `peek()`:

```
class FoundationPile {
  boolean isEmpty() { ... }
  /*
   * @return The card on top of the pile.
   * @throws EmptyStackException if isEmpty()
   */
  Card peek() {
    if (isEmpty()) {
      throw new EmptyStackException();
    }
    /* ... */
  }
}
```

In this case, calling `peek` on an empty pile *should* result in an `EmptyStackException`. This is part of the specified, expected behavior. If no exception is raised when called on an empty pile, then the `peek()` method does not do what is expected, and this means it is faulty. We should have a test to detect this potential fault.

With JUnit 5, the standard way to check that a method call raises the expected exception is to use the `assertThrows` assert method. This method takes as argument an instance of class `Class` that represents the expected exception type, and an instance of a subtype of the library interface `Executable` that executes the code expected to cause the exception. This programming idiom thus requires the use of a function object, as described in Section 3.4.

```
@Test
void testPeek_Empty() {
  assertThrows(EmptyStackException.class, new Executable() {
    public void execute() throws Throwable {
      aPile.peek();
    }
  });
}
```

In this example, I used an anonymous class to define the behavior of method `execute` which, in this case, calls `peek` on an empty pile. The reason this seemingly complicated setup is required is that we need our exception-causing code to execute *within* the execution of `assertThrows`. For this reason, we pass in a function object of type `Executable`. When `assertThrows` executes, it calls `execute()` on its second argument, which normally triggers the exception.

In practice, lambda expressions are typically used with `assertThrows`. Lambda expressions were briefly introduced in Section 3.4, and will be covered in detail in Chapter 9. With a lambda expression, the use of `assertThrows` is much more compact:

```
@Test
void testPeek_Empty_LambdaExpression() {
  assertThrows(EmptyStackException.class, () -> aPile.peek());
}
```

Code Exploration: JetUML · TestDiagramType

Testing exception handling behavior.

Class `TestDiagramType` contains examples of the use of `assertThrows`. According to the specification of method `fromName`, if the user attempts to retrieve an invalid diagram type, the test will check that an `IllegalArgumentException` is raised as expected. Looking at the implementation of method `fromName`, one would notice that it is robust enough to handle a null input in exactly the same way as an invalid non-null input. Why write two distinct tests, then? I made this decision to clarify the behavior for null inputs and to guard against future modifications that could alter this behavior.

5.7 Encapsulation and Unit Testing

If we locate unit tests in the same package as the code they are testing, as described in Section 5.3, then the tests have access to all fields and methods of the corresponding classes, except for private members. However, there are situations where test code may need access to private fields or methods, either to write assertions verifying the outcome of a test, or to test private methods.

For example, let us consider that the interface to class `FoundationPile` (see Section 5.5) does not include a method `size()` to return the number of cards in the pile. Presumably, because `FoundationPile` does not have a `size()` method, and following the principle of information hiding, we can assume that no part of the production code needs this information. However, in unit tests, we might want to check that the size of the pile changes as we push cards onto it. As for testing private methods, a major obstacle is quite simply that we cannot call private methods of other classes directly from test code. How can we deal with such situations? There are three main approaches to accessing private members in the context of testing.

A first approach, which I refer to as the *hard line*, is to strictly respect the tight encapsulation of the design and to work around it to the extent possible. In some cases, it might be possible to access some of the private state of an object indirectly through the accessible methods. For example, in the case of the `FoundationFile` mentioned above, we can discover the size of the pile by popping all cards from a pile into a list and then restoring the cards into the pile. In our testing code, we can thus implement our own *helper method* to obtain the size of a pile:

```
private static int size(FoundationPile pPile) {
  List<Card> cards = new ArrayList<>();
  while (!pPile.isEmpty()) {
    cards.add(pPile.pop());
  }
  int size = cards.size();
  while (!cards.isEmpty()) {
    pPile.push(cards.removeLast());
  }
  return size;
}
```

In fact, if the testing class needs only a single `FoundationPile` instance as a target object for testing, then it is possible to make this instance part of the scaffolding and convert our helper method into an instance method of the test class (by referring to a field `aPile` instead of a formal parameter). Either way, we can now learn about the state of the object without weakening its encapsulation. With the hard line approach, however, it is simply not possible to test private methods directly. Instead, private methods have to be tested indirectly, by virtue of testing the accessible methods that rely on them. Some experts argue that this is the proper approach, because private methods are internal elements of other, accessible methods, and therefore are not really units that should be tested. However, this hard line approach results in a loss of opportunities for testing small methods in isolation. There can exist situations where some nontrivial computation is neatly abstracted in a method restricted to a class's scope (i.e., made private), yet it remains worthwhile to ensure it meets our expectations.

A second approach is to *weaken the encapsulation* of the production code for the purpose of testing. This would typically mean removing the `private` access modifier for methods, and thus giving the necessary method default visibility, which makes them accessible to all other classes within the same package (see Section 2.3). In our example, this could mean adding a new method `size()` with default visibility to the `FoundationPile` class, along with removing the `private` modifier for any method that should be tested in isolation. Methods made *visible for testing* should be documented as such, ideally by using an annotation (see Section 5.4). Unfortunately, weakening the quality of the encapsulation for the purpose of testing has a negative impact on the quality of the encapsulation as a whole. Unless the class members made visible for testing are carefully identified and only referred to from test code, the risk is that other classes may come to depend on them, introducing additional dependencies and making the design more fragile and difficult to understand.

A third approach is to *use metaprogramming* to get around access restrictions in test code. For the sake of discussion, let us assume that class `FoundationPile` also has a private method `getSuit()` that returns the suit of the cards in the pile, and that we would like to test it separately.

```
private Suit getSuit() {
  assert !isEmpty();
  return aPile.getFirst().suit();
}
```

In our test code, we create a helper method that calls this private method using metaprogramming.

```
public class TestFoundationPile {
  private FoundationPile aPile = new FoundationPile();

  private Suit executeGetSuit() {
    try {
      Method method =
        FoundationPile.class.getDeclaredMethod("getSuit");
      method.setAccessible(true);
      return (Suit) method.invoke(aPile);
    }
    catch (ReflectiveOperationException exception) {
      throw new AssertionError("Reflection error");
    }
  }

  @Test
  void testGetSuit() {
    aPile.push(ACE_CLUBS);
    assertSame(Suit.CLUBS, executeGetSuit());
  }
}
```

In the test class, we define a helper method `executeGetSuit` that launches the execution of the unit under test (`getSuit()`) on an instance `aPile`, which forms part of the test fixture. With this helper method, the code of the corresponding test looks mostly normal. However, there is one big difference: the call to `executeGetSuit()` is not a direct call to the unit under test, `getSuit()`. Instead, the call is to a helper method `executeGetSuit()` that uses metaprogramming to call the unit under test while bypassing the access restriction of the `private` keyword. In contrast to weakening encapsulation by removing the `private` keyword, metaprogramming enables access to the private method only at one specific point in the testing code, which eliminates the risk that other production code can be made to depend on the method of interest. However, this approach results in brittle tests that can break if the associated code is refactored. For this reason, it is best employed only when the benefits are significant.

Code Exploration: Solitaire · TableauTest

Using metaprogramming to access private structures.

Class `Tableau` represents the seven piles of cards that fan downwards in a game of Solitaire. It defines a private method `getPreviousCard(Card)` that returns the card stacked under the input card in a given pile. Class `TableauTest` provides a working implementation of an invocation of `getPreviousCard` using metaprogramming, along with examples of tests that access the private method in this way.

5.8 Testing with Stubs

The key to unit testing is to test small parts of the code *in isolation*. In some cases, however, factors can make it difficult to test a piece of code in isolation, for example, when the part we want to test:

- triggers the execution of a large chunk of other code;
- includes sections whose behavior depends on the environment (e.g., system fonts);
- involves non-deterministic behavior (e.g., randomness).

Such a case is illustrated in the following design, which is a simplified version of the Solitaire example application. The `GameModel` class has a `tryToAutoPlay()` method that triggers the computation of the next move by delegating the task to a strategy which, due to polymorphism, could be any of a number of options (see Figure 5.5). Here we would like to write a unit test for the `GameModel#tryToAutoPlay` method.

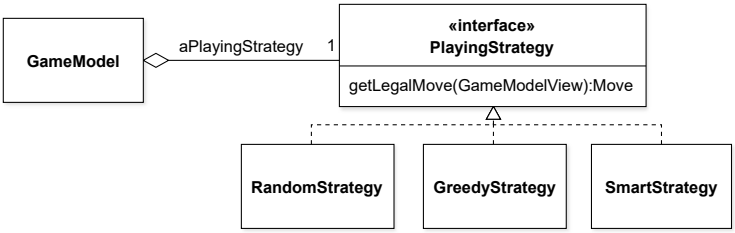


Fig. 5.5 Playing strategies in `GameModel`

In this task we face at least four problems:

- Calling the `tryToAutoPlay` method on an instance of `GameModel` will delegate the call to `getLegalMove` on a strategy object, which will involve the execution of presumably complex behavior to realize the strategy. This does not align well with the concept of unit testing, where we want to test small pieces of code in isolation.
- The implementation of the strategy may involve some randomness.
- We do not know which strategy would be used by the game engine. Yet, we need to determine an oracle for the results.
- It is unclear how this is different from testing the strategies individually.

The way out of this conundrum is to realize that the responsibility of `GameModel#tryToAutoPlay` is not to compute the next move, but rather to delegate this to a strategy. So, to write a unit test that tests that the UUT does what it is expected to do, we only need to assert that it properly relays the request to obtain a move to a strategy. This can be achieved with the help of a *stub*.

A stub is a simplified version of an object that mimics its behavior sufficiently to support the testing of a UUT that uses this object. Typically, a stub will also contain some logging or other instrumentation code to provide information to the test that can assist in completing the assertion. To use stubs, we also need a supertype for classes we wish to create stub objects for. In our case, this is the interface type `PlayingStrategy`. Continuing with our `tryToAutoPlay` situation, we start by defining a stub for the strategy:

```
public class GameModelTest {
  static class StubStrategy implements PlayingStrategy {
    private boolean aExecuted = false;

    public boolean hasExecuted() {
      return aExecuted;
    }

    public Move getLegalMove(GameModelView pModelView) {
      aExecuted = true;
      return new NullMove();
    }
  }
}
```

This strategy does nothing except remember that its `getLegalMove` method has been called, and returns a `Move` object of type `NullMove` (an application of the NULL OBJECT pattern). We can then use an instance of this stub instead of a real strategy in the test. The challenge at this point is to initialize our game model with a stub instead of a full-featured strategy. This task will be greatly facilitated by the use of dependency injection in the overall design (see Section 3.8). Let us assume that we designed the `GameModel` class to make it possible to inject the desired strategy when initializing the class:

```
public class GameModel implements GameModelView {
  private final PlayingStrategy aPlayingStrategy;

  public GameModel(PlayingStrategy pStrategy) {
    aPlayingStrategy = pStrategy;
  }

  public void tryToAutoPlay() {
    aPlayingStrategy.getLegalMove(this);
  }
  ...
}
```

Then injecting the stub and using it for the assertion is straightforward.

```
@Test
void testTryToAutoPlay() {
  StubStrategy stub = new StubStrategy();
  GameModel model = new GameModel(stub);
  model.tryToAutoPlay();
  assertTrue(stub.hasExecuted());
}
```

In addition to demonstrating the use of stubs for testing, the above example illustrates the benefits of limiting coupling between classes. The fact that `GameModel` does not depend on any concrete playing strategy makes it easier to understand and to test, because we can focus solely on the logic of the `GameModel`. If the `GameModel` class had *not* provided a mechanism for injecting a strategy, testing the `tryToAutoPlay` method would have been much more difficult. One option would have been to rely on reflection to inject the stub but, as discussed in Section 5.7, this method is brittle and not generally recommended. In some cases, it can even be worth it to refactor code to improve its testability, for example by replacing hard-coded dependencies by an injection mechanism. This section provided only an introduction to the use of stubs for testing. In practice, the use of stubs can get very sophisticated, and frameworks exist to support this task if necessary.

5.9 Test Coverage

Up to now this chapter covered how to define and structure unit tests, but avoided the question of what inputs to provide to the unit under test. Notwithstanding the example of exhaustive testing in Section 5.1, it should be clear that, for the majority of UUTs, it is not even physically possible to exhaustively test the input space. For example, as discussed in Section 4.2, the number of different arrangements of cards that an instance of class `Deck` can take is astronomical (2.2×10^{68}). Even with cutting-edge hardware, testing any of the methods of class `Deck` for all possible inputs (i.e., possible states of the implicit parameter) would take an amount of time many times greater than the age of the universe. This is quite incompatible with the requirement that unit tests execute quickly (see Section 5.5).

Hence, we need to *select* some input out of all the possibilities. This is a problem known as *test case selection*, where *test case* can be considered to be a set of input values for a UUT. For example, an instance of `Deck` with a single Ace of Clubs in the deck is a test case of the method `Deck.draw()`. The challenge of the test case selection problem is to test *efficiently*, meaning to find a minimal set of test cases that provides us a maximal amount of testing for our code. Unfortunately, while it is fairly intuitive what a minimal number of test cases is, there is no natural or even agreed-upon definition of what an *amount of testing* is. However, there is a large body of research and practical experience on the topic of test case selection (see Further Reading for a recommendation). In this section, I only summarize the key theoretical tenets and practical insights necessary to get started with test case selection. There are two basic ways to approach the selection of test cases:

- **Functional (or black-box) testing** tries to cover as much of the *specified* behavior of a UUT as possible, based on some external specification of what the UUT should do. For the `Deck.draw()` method, this specification is that the method should result in the top card of the deck being removed and returned. There are many advantages to black-box testing, including that it is not necessary to access

the code of the UUT, that tests can reveal problems with the specification, and that tests can reveal missing logic.

- **Structural (or white-box) testing** tries to cover as much of the *implemented* behavior of the UUT as possible, based on an analysis of the source code of the UUT. An example is provided below. The main advantage of white-box testing is that it can reveal problems caused by low-level implementation details that are invisible at the level of the specification.

Functional testing involves selecting values at meaningful boundaries in the input range given the documented behavior of the UUT. In many cases, this process is relatively intuitive (for example, selecting a negative, zero, and positive value for integer inputs, testing with empty data structures, etc.). Coverage of functional testing techniques beyond this basis is outside of the scope of this book, so the remainder of this section provides a review of the main concepts of structural testing. Let us consider again the implementation of the `canMoveTo` method of class `FoundationPile` (see Section 5.5), with the `assert` statement removed to simplify the discussion).

```
boolean canMoveTo(Card pCard) {
  if (isEmpty()) {
    return pCard.rank() == Rank.ACE;
  }
  else {
    return pCard.suit() == peek().suit() &&
      pCard.rank().ordinal() == peek().rank().ordinal() + 1;
  }
}
```

We can intuitively see that the code structure can be partitioned into different parts that might be good to test. First, there is the case where the pile is empty (the `true` part of the `if` statement), and the case where it is not empty (the `else` block). But then, each of these parts can also be partitioned into different sub-parts, for example to cover the case where the cards are in the correct sequence, but of different suits. In the general case, things can get hairy, and it is easy to get lost without a systematic way to reason about the code.

One common method for determining what to test is based on the concept of *coverage*. A *test coverage metric* is a number (typically a percentage) that determines how much of the code executes when we run our tests. Test coverage metrics can be computed by code coverage tools that keep track of the code that gets executed when we run unit tests. This sounds simple, but the catch is that there are different definitions of what we can mean by *code*, in the context of testing. Each definition is a different way to compute *how much testing* is done. Certain software development organizations may have well-defined *test adequacy criteria* whereby test suites must meet certain coverage thresholds, but in other cases, the insights provided by coverage metrics are used more generally to help determine where to invest future testing efforts. The following are three well-known coverage metrics (there are many others, see Further Reading).

Statement Coverage

Let us start with the simplest coverage metric: *statement coverage*. Statement coverage is the number of statements executed by a test or test suite, divided by the number of statements in the code of interest. Given the implementation of `canMoveTo` shown above, the following test:

```
@Test
void testCanMoveTo_Empty_ReturnsFalse() {
  assertFalse(emptyPile.canMoveTo(THREE_CLUBS));
}
```

achieves $2/3 = 67\%$ coverage, because the conditional statement predicate and the single statement in the `true` branch are executed, and the single statement in the `false` branch is not. The logic behind statement coverage is that if a fault is present in a statement that is never executed, the tests are not going to help find it. Although this logic may seem appealing, statement coverage is actually a poor coverage metric. A first reason is that it depends on the detailed structure of the code. We could rewrite the `canMoveTo` method as follows, and achieve 100% test coverage with exactly the same test.

```
boolean canMoveTo(Card pCard) {
  boolean result = pCard.suit() == peek().suit() &&
    pCard.rank().ordinal() == peek().rank().ordinal()+1;
  if (isEmpty()) {
    result = pCard.rank() == Rank.ACE;
  }
  return result;
}
```

The second reason is that not all statements are created equally, and there can be quite a bit that goes on in a statement if this statement involves a compound Boolean expression (as is the case of the first statement in the last example).

Branch Coverage

Branch coverage is the number of program *branches* executed by the test(s) divided by the total number of branches in the code of interest. In this context, a branch is one of the two possible outcomes of a *condition* (a decision point). Branch coverage is a stronger metric than statement coverage in the sense that for the same coverage result, more of the possible program executions will have been tested. Unfortunately, the concept of branch coverage is ambiguous, due to the different possible interpretations of the term *branch*. In the code of `canMoveTo`, there are only two branches if we only consider the single `if` statement. However, both the `true` and the `false` branches lead to statements that consist of Boolean expressions, which are themselves another type of condition because they have to possible outcomes (`true` or `false`). To be consistent with popular coverage analysis tools, I adopt the definition that Boolean expressions within statements also introduce branches. Although

more complex to determine, this definition is also more useful. With this definition, the original code of `canMoveTo` exhibits four conditions: the `if`, the first `return` statement, the first comparison in the second return statement, and the second comparison in the second return statement. Each of these four condition has a `true` and `false` branch, for a total of eight branches. The only test written so far thus has only $2/8 = 25\%$ branch coverage. If we add the other two tests shown in Section 5.5 to the test suite:

```
@Test
void testCanMoveTo_Empty_ReturnsTrue() {
  assertTrue(aPile.canMoveTo(ACE_CLUBS));
}

@Test
void testCanMoveTo_NotEmptyAndSameSuit_ReturnsFalse() {
  aPile.push(ACE_CLUBS);
  assertFalse(aPile.canMoveTo(THREE_CLUBS));
}
```

we get $6/8 = 75\%$. This is pretty good, but our systematic coverage analysis points out that we are actually missing two branches. One of the missing branches is the case where the input card has the same suit as the pile and whose rank is immediately above the rank of the card at the top of the pile. The second missing branch is the case where the input card is not of the same suit as the cards in the pile. Branch coverage is one of the most useful test coverage criteria. It is well supported by testing tools and relatively straightforward to interpret, and also *subsumes* statement coverage, meaning that achieving complete branch coverage implies complete statement coverage.

Path Coverage

There are other coverage metrics stronger than branch coverage. For example, one could, in principle, compute a *path coverage* metric as the number of execution paths actually executed over all possible execution paths in the code of interest. Path coverage subsumes almost all other coverage metrics, and is a very close approximation of the entire behavior that is possible to test. Unfortunately, in many cases, the number of paths through a piece of code will be unbounded, so it will not be possible to compute this metric. For this reason, path coverage is considered a theoretical metric, useful for reasoning about test coverage in the abstract, but without any serious hope of general practical applicability. Interestingly, the number of paths in the code of `canMoveTo` is actually only five, so, less than the number of branches! The path coverage of the three-test test suite above can thus be computed, at $3/5 = 60\%$. Because the structure of the code is without loops, this is not overly surprising. However, as soon as loops enter the picture, reasoning about paths becomes troublesome.

Insights

This chapter described techniques to structure and implement unit tests for a project, and argued that unit tests can provide valuable feedback on the design of production code. The following insights assume you have decided to adopt unit testing and are using a unit testing framework.

- Every unit test includes the execution of a unit under test (UUT), input data passed to the UUT, an oracle that describes what the result of executing the UUT should be, and one or more assertions that compare the result of the execution with the oracle;
- Design your unit tests so that they are focused, that is, that they isolate and test a small and well-defined amount of behavior;
- Design your unit tests to run fast, be independent from each other, and be repeatable in any computing environment;
- Design your unit tests to be readable: consider using the name of the test and local variables to add clarity about what you are testing and to describe the oracle;
- Organize your test suite cleanly, with a clear mapping between tests and the code units they test. Consider separate source code directories with a parallel package structure for production and test code;
- Metaprogramming is a powerful language feature that allows you to write code to analyze other code. However, it must be used with care in production code, because it is prone to run-time errors;
- Type annotations can provide metadata about certain program elements, which can then be accessed through metaprogramming;
- Use test fixtures to structure your testing code cleanly. Remember that tests should not make any assumptions about the order in which they are executed;
- Do not test for unspecified behavior, and in particular for input that does not respect a method's preconditions;
- Exceptions that can be raised are often an explicit part of a method's interface, in which case the raising of exceptions constitutes behavior that can be tested;
- Try to avoid weakening the interface of a class only for the purpose of testing. Instead, consider writing helper methods in the test class to obtain this information. If you must make class members visible for testing, document this decision with annotations. An alternative is to use metaprogramming;
- To isolate the behavior of stateful objects that refer to many other objects, consider using stubs to model the behavior of the component objects and provide instrumentation to provide data about the execution of the stubs;
- Use test coverage metrics to reason about how much of the program's behavior you are testing. Favor branch coverage over statement coverage;
- Remember that having tests that pass does not guarantee that the code is correct.

Further Reading

The book *Effective Software Testing: A Developer's Guide* by Maurício Aniche [1] provides a comprehensive treatment of testing, including the definition of many test coverage metrics. The Java Tutorial [11] provides a good introduction to annotations and reflection (Java's version of metaprogramming). Documentation on how to use JUnit is available on the JUnit website.